

Bitácora completa — Laboratorio No. 02

OS Setup, Shell y Software de Soporte de Red

Camilo Aguirre · Juan David Rangel

Arquitectura y Servicios de Red (AYSR) — Escuela Colombiana de Ingeniería Julio
Garavito

*Todo lo que hicimos, comando por comando, con explicación y ejemplos —
para tener a mano en la sustentación.*

Índice

1. Packet Tracer — topología, dispositivos y cables
2. Direccionamiento IP y protocolos de la topología (VTP, OSPF)
3. Comandos IOS usados (Cisco CLI)
4. Tracking Messages — ping, ARP y encapsulación
5. Wireshark — interfaz, filtros y hallazgos
6. Scripts de shell — código completo línea por línea
7. Editor VI — comandos completos
8. SAMBA / SMB — configuración y comandos
9. Despliegue de VMs y redes
10. Conceptos raros / preguntas trampa

1. Packet Tracer — topología, dispositivos y cables

La red se armó en Packet Tracer 9.0.1 con la topología completa que pide el enunciado: 4 routers, una serie de switches (uno de ellos multicapa) y varios PCs/laptops y servidores. Así es como está conectada realmente (según el archivo `diagrama-camilo-final.pkt`):

Dispositivo	Modelo	Conectado a	Tipo de cable
Router3	1941	Laptop5 · Router1	Ethernet (Laptop) / Serial (Router1)
Router1	1941	Router3 · Router0 · Switch3	Serial (routers) / Ethernet (Switch3)
Router0	1841	Router1 · Router2 · Multilayer Switch0	Serial (routers) / Ethernet (switch)
Router2	1841	Router0 · Switch4	Serial (Router0) / Ethernet (Switch4)
Multilayer Switch0	3560-24PS	Router0 · Switch1 · Switch2 · Laptop4	Ethernet
Switch1 / Switch2	2950-24	Multilayer Switch0 · su laptop	Ethernet (troncal VTP client/transparent)
Switch3	2950-24	Router1 · Laptop0, Laptop1, Laptop3	Ethernet
Switch4	2950-24	Router2 · Server RADIUS, Server SYSLOG, Server DHCP	Ethernet

Cómo se arma paso a paso en Packet Tracer

1. Del panel inferior de dispositivos se arrastra cada router/switch/PC al área de trabajo (categorías: Routers, Switches, End Devices).
2. Se selecciona la herramienta de conexiones (ícono de rayo) y se elige el cable correcto:
 - **Copper Straight-Through** (Ethernet normal): PC↔Switch, Switch↔Router, Switch↔Switch cuando son de igual "tipo" de puerto. Es el cable negro sólido del diagrama.
 - **Serial DCE/DTE** (rojo): para los enlaces WAN entre routers (Router3–Router1, Router1–Router0, Router0–Router2). Un extremo es DCE (define el clock rate con `clock rate 64000`) y el otro DTE.
3. Al conectar, aparecen puntos en cada extremo del cable: **verde** = enlace activo (Up), **ámbar** = negociando (STP en blocking/listening), **rojo** = caído (Down). Verificamos que todos quedaran en verde antes de configurar IPs (ver Figura 1).

4. Se configura cada interfaz desde la pestaña *Config* (modo gráfico) o la *CLI* (modo comando, igual que un router real).

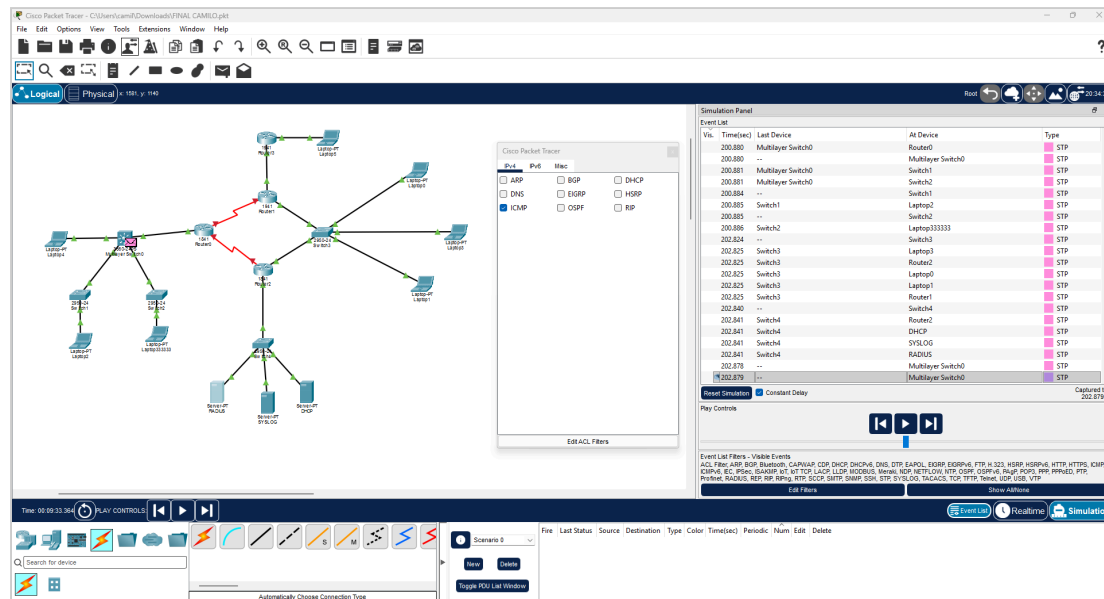


Figura 1. Topología completa corriendo en Simulation mode — todos los enlaces en verde (Up), incluido el serial Router0–Router2.

¿Qué significan los puntos negros/blancos en las conexiones del diagrama del enunciado?

Sólido = Ethernet físico directo (capa 1/2 real). Punteado = conexión lógica/virtual (sin cable físico directo entre esos dos puntos concretos — por ejemplo, una ruta a través de la red o un enlace inalámbrico). No indican si el link está Up o Down; eso lo muestran los puntos verdes/rojos en los extremos del cable.

2. Direccionamiento IP y protocolos de la topología

2.1 Esquema de IPs

Segmento	Red	Detalle
WAN Router0–Router1	7.0.0.0/30	R0=.1, R1=.2
WAN Router1–Router2	8.0.0.0/30	R1=.1, R2=.2
LAN Router0 (VLAN10)	10.0.10.0/24	gw .1; servidores .10/.11; switch .2
LAN Router0 (VLAN30)	10.0.30.0/24	gw .1
LAN Router2	10.0.20.0/24	gw .1; Server RADIUS/SYSLOG/DHCP

¿Por qué /30 en los enlaces entre routers?

Un enlace punto a punto (WAN serial) solo necesita 2 direcciones utilizables (una por cada extremo del router). Una /30 da 4 direcciones totales: red, 2 utilizables, broadcast — es la máscara más chica que alcanza para 2 hosts, y no se desperdician IPs de una red más grande.

2.2 VTP (VLAN Trunking Protocol)

La topología usa un dominio VTP llamado `facil`. El Multilayer Switch0 actúa como **VTP server** (crea/edita VLANs y las propaga), y Switch1/Switch2 son **VTP client** / **VTP transparent** según el diagrama del enunciado.

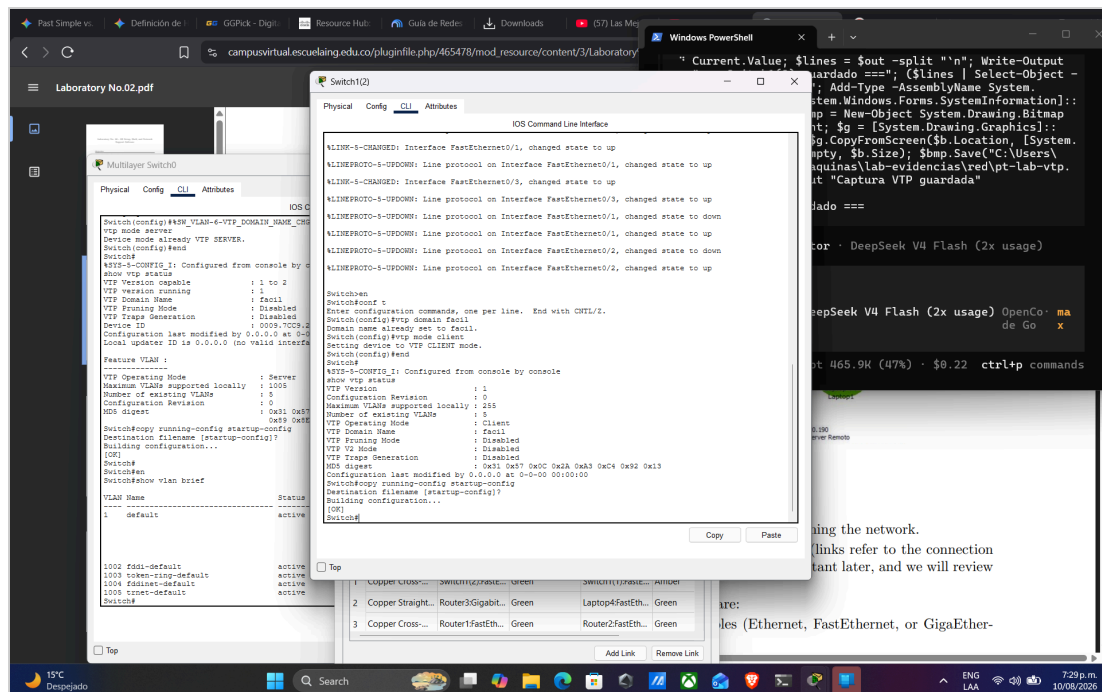


Figura 2. Salida real de `show vtp status` en uno de los switches: dominio "facil", modo Server/Client, 5 VLANs existentes, número de revisión de configuración.

```
Switch(config)#vtp domain facil
Domain name already set to facil.
Switch(config)#vtp mode server
Device mode already VTP SERVER.
Switch#show vtp status
VTP Version capable      : 1 to 2
VTP version running      : 1
VTP Domain Name         : facil
VTP Pruning Mode        : Disabled
VTP Traps Generation     : Disabled
Number of existing VLANs : 5
Configuration Revision   : 1
```

VTP server / client / transparent — la pregunta más típica.

Server: puede crear, borrar y modificar VLANs; las anuncia a todo el dominio.

Client: recibe las VLANs que anuncia el server, pero no puede crearlas localmente.

Transparent: no participa del dominio VTP — reenvía los anuncios pero mantiene sus propias VLANs locales, independientes. El *número de revisión* es clave: si conectas un switch con una revisión más alta (aunque tenga configuración vieja/basura) puede sobrescribir el dominio entero — por eso siempre se pone un switch nuevo en modo transparent antes de conectarlo.

2.3 OSPF (Open Shortest Path First)

Los routers corren OSPF con **process-id 130** (número local, no tiene que coincidir entre routers, solo identifica el proceso dentro del mismo router) dentro del **area 0** (área backbone).

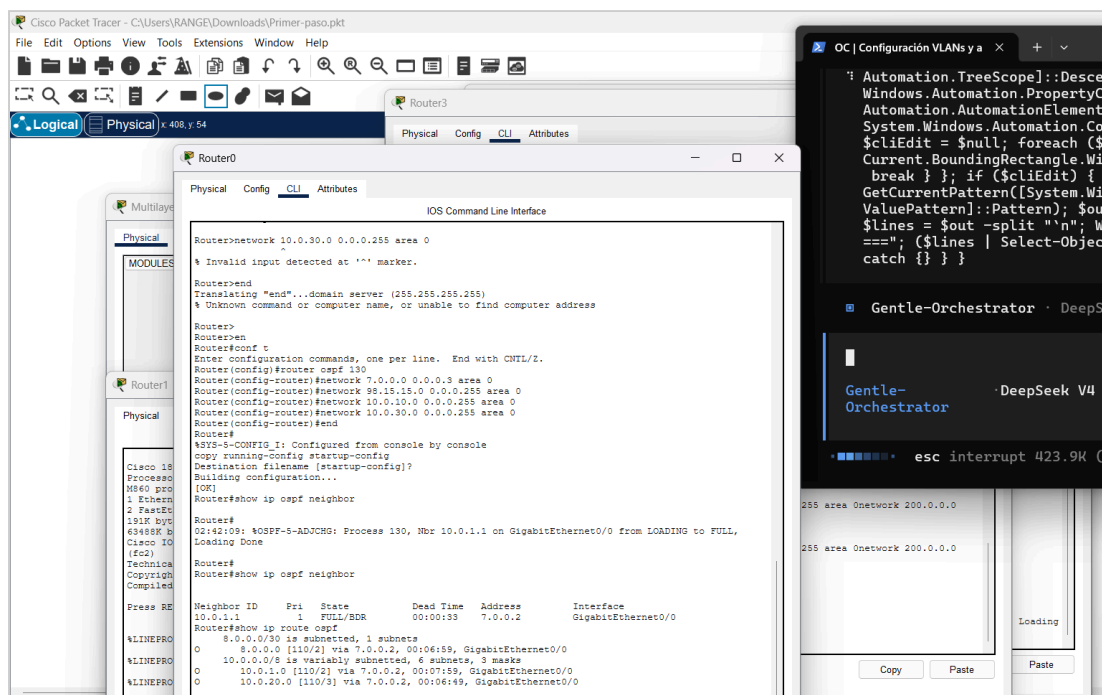


Figura 3. Configuración y verificación real de OSPF en uno de los routers: `router ospf 130`, redes anunciadas con wildcard mask, vecino OSPF en estado FULL, tabla de rutas OSPF.

```
Router(config)#router ospf 130
Router(config-router)#network 7.0.0.0 0.0.0.3 area 0
Router(config-router)#network 98.15.15.0 0.0.0.255 area 0
Router(config-router)#network 10.0.10.0 0.0.0.255 area 0
Router(config-router)#network 10.0.30.0 0.0.0.255 area 0
Router(config-router)#end

Router#show ip ospf neighbor
Neighbor ID      Pri   State           Dead Time   Address      Interface
10.0.1.1         1     FULL/BDR        00:00:33    7.0.0.2     GigabitEthernet0/0

Router#show ip route ospf
      8.0.0.0/30 is subnetted, 1 subnets
O       8.0.0.0 [110/2] via 7.0.0.2, 00:06:59, GigabitEthernet0/0
      10.0.0.0/8 is variably subnetted, 6 subnets, 3 masks
O       10.0.1.0 [110/2] via 7.0.0.2, 00:07:59, GigabitEthernet0/0
O       10.0.20.0 [110/3] via 7.0.0.2, 00:06:49, GigabitEthernet0/0
```

¿Qué es la wildcard mask (0.0.0.3, 0.0.0.255)?

Es la máscara "invertida" que usa OSPF (y las ACLs) en vez de la máscara de subred normal. Se calcula restando cada octeto de la máscara a 255: máscara 255.255.255.252 → wildcard 0.0.0.3; máscara 255.255.255.0 → wildcard 0.0.0.255. Un 0 en la wildcard significa "ese octeto tiene que coincidir exacto"; un

255 significa "no me importa ese octeto".

¿Qué es el "110" entre corchetes en `show ip route`?

Es la *distancia administrativa* de OSPF (110) — qué tan "confiable" es la fuente de la ruta. El número después de la barra ([110/2]) es el *costo* OSPF hacia esa red (menor costo = mejor ruta, se basa en el ancho de banda de las interfaces).

¿Qué significa el estado `FULL/BDR` de un vecino OSPF?

`FULL` = la adyacencia OSPF está completamente formada, las bases de datos de topología están sincronizadas. `BDR` = ese vecino es el Backup Designated Router del segmento (rol que OSPF elige en redes multiacceso para reducir la cantidad de adyacencias).

3. Comandos IOS usados (Cisco CLI)

Comando	Explicación
<code>enable</code> (o <code>en</code>)	Pasa de modo usuario (<code>Router></code>) a modo privilegiado (<code>Router#</code>), que permite ver/cambiar configuración.
<code>configure terminal</code> (<code>conf t</code>)	Entra al modo de configuración global (<code>Router(config)#</code>).
<code>interface gi0/0</code>	Entra a configurar una interfaz específica.
<code>ip address 10.0.10.1 255.255.255.0</code>	Asigna IP y máscara a la interfaz actual.
<code>no shutdown</code>	Activa la interfaz (por defecto las interfaces de router vienen administrativamente apagadas).
<code>router ospf 130</code>	Activa el proceso OSPF número 130 en este router.
<code>network X.X.X.X W.W.W.W area 0</code>	Le dice a OSPF qué redes anunciar, con wildcard mask, en el área 0.
<code>show ip ospf neighbor</code>	Muestra los vecinos OSPF y su estado de adyacencia.
<code>show ip route</code> / <code>show ip route ospf</code>	Tabla de rutas completa / solo las aprendidas por OSPF.
<code>copy running-config startup-config</code>	Guarda la configuración activa (en RAM) al arranque (NVRAM), para que sobreviva un reinicio.
<code>vtp domain facil</code>	Fija el nombre del dominio VTP.
<code>vtp mode server / client / transparent</code>	Fija el rol VTP del switch.
<code>show vtp status</code>	Muestra dominio, modo, número de revisión y VLANs existentes.
<code>show vlan brief</code>	Lista las VLANs configuradas y qué puertos pertenecen a cada una.
<code>ping <ip></code>	Prueba de conectividad ICMP desde la CLI del router/PC.
<code>show ip interface brief</code>	Resumen rápido de IP y estado (up/down) de todas las interfaces.

4. Tracking Messages — ping, ARP y encapsulación

Se activa el modo **Simulation** (esquina inferior derecha de Packet Tracer), se abre **Edit Filters** y se dejan marcados solo **ICMP** y **ARP** (para no ver de fondo el ruido de STP/DTP que generan los switches al converger). Desde la consola del servidor RADIUS se ejecuta `ping <IP del DHCP>` y se corre **Auto Capture / Play** hasta "No More Events".

Secuencia esperada

1. **ARP Request** (broadcast, MAC destino `FF:FF:FF:FF:FF:FF`): "¿quién tiene esta IP? decime tu MAC".
2. **ARP Reply**: el dueño de la IP responde con su MAC real.
3. **ICMP Echo Request** (Type 8, Code 0): el ping en sí.
4. **ICMP Echo Reply** (Type 0): la respuesta.

Encapsulación del paquete ICMP, capa por capa

Capa	Qué contiene
Ethernet (L2)	MAC origen / MAC destino, EtherType <code>0x0800</code> (IPv4)
IP (L3)	IP origen / IP destino, Protocol <code>1</code> (ICMP), TTL 128
ICMP (L4)	Type 8/o, Code 0, número de secuencia

¿Por qué la MAC cambia salto a salto pero la IP no?

La IP es la dirección *lógica de extremo a extremo*: identifica origen y destino final del paquete, y no cambia mientras viaja por la red (salvo NAT). La MAC es la dirección *física del enlace actual*: en cada salto de router, la trama Ethernet se reconstruye con la MAC del siguiente dispositivo en la ruta — por eso decimos que "IP es global, MAC es local".

5. Wireshark — interfaz, filtros y hallazgos

5.1 Los 3 paneles

- **Lista de paquetes** (arriba): una fila por paquete — número, tiempo, IP origen/destino, protocolo, resumen.
- **Detalle del paquete** (medio): árbol expandible con cada capa (Frame, Ethernet, IP, TCP, HTTP...).
- **Bytes** (abajo): el contenido crudo en hexadecimal + ASCII.

5.2 Filtros usados (y su sintaxis)

Filtro	Qué muestra
<code>ip.addr == 192.168.1.6</code>	Todo el tráfico donde esa IP es origen O destino
<code>tcp.port == 443</code>	Tráfico HTTPS (TLS)
<code>http.request.method == "GET"</code>	Solo peticiones GET
<code>icmp</code>	Solo pings

Un filtro de Wireshark NO borra paquetes, los oculta. Se pueden quitar en cualquier momento y la captura completa sigue intacta en memoria/archivo.

5.3 Captura real y hallazgos

Captura de 30 s a `www.scielo.org.co` (HTTP, sin cifrar a propósito): **2.742 paquetes** / 3.004.210 bytes. Host `192.168.1.6`, gateway `192.168.1.1` (MAC `e0:a1:ce:d2:85:76`), NIC `08:f9:7e:9e:25:03`.

- DNS: 2 paquetes (consulta + respuesta) → resuelve a `168.176.28.57`.
- Handshake TCP (SYN, SYN-ACK, ACK): frames 361–394.
- Petición `GET / HTTP/1.1`: frame **395**. Respuesta HTTP 200, 9.200 bytes.
- RTT promedio 150 ms.
- La conversación con SciELO fue solo 36 paquetes de los 2.742 totales — el resto (streaming de Google, TLS, QUIC) se capturó porque la NIC estaba en **modo promiscuo**.
- 17 retransmisiones (0,6%) — normal en Wi-Fi; 14 ACKs duplicados; o congestión.

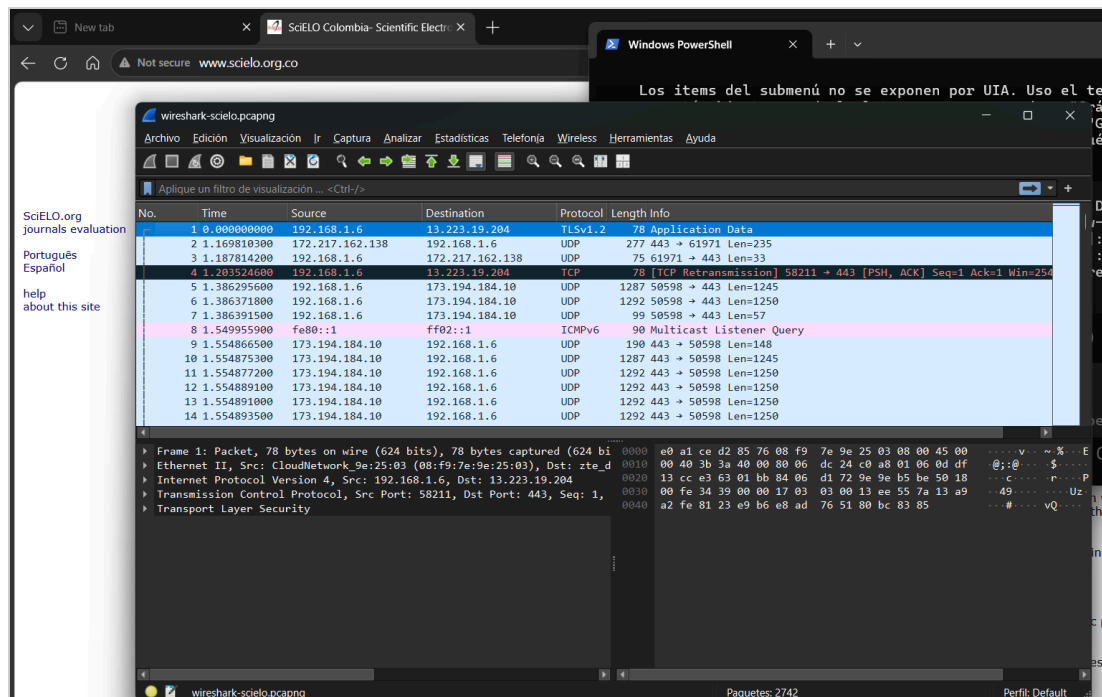


Figura 4. Interfaz de Wireshark con la captura real cargada.

Modo promiscuo. Normalmente la NIC descarta las tramas que no van dirigidas a su propia MAC (o broadcast/multicast) — las ignora a nivel de hardware. En modo promiscuo, la tarjeta entrega TODAS las tramas que le llegan al sistema operativo, vayan o no dirigidas a ella. Por eso una captura en un Wi-Fi doméstico "ve" tráfico de otros dispositivos del mismo segmento.

6. Scripts de shell — código completo línea por línea

Los 4 ejercicios se resolvieron en Slackware 15.0, guardados en `/root/scripts/` del repo (carpeta `maquinas/scripts-lab02/`). Se muestra el código completo de cada uno, y abajo de cada uno una tabla que explica **línea por línea, en criollo**, qué hace — para poder sustentarlo sin tener que memorizar nada de un tirón.

Mini-diccionario — lo único que hay que tener claro antes de leer los scripts

Se ve así	Qué es
<code>#!/bin/bash</code>	La primera línea de TODO script: le dice al sistema "ejecutame usando el programa bash". Sin esto, el sistema no sabe con qué intérprete correr el archivo.
<code># texto</code>	Un comentario. La máquina lo ignora por completo; es una nota para el humano que lee el código.
<code>echo "..."</code>	Escribe ese texto en pantalla. Es el "print" de bash.
<code>read -p "..."</code> <code>variable</code>	Muestra el texto como pregunta, se queda esperando a que el usuario escriba algo y le dé Enter, y guarda lo que escribió dentro de <code>variable</code> .
<code>\$variable</code>	"El valor que está guardado ahí". Si <code>dir</code> vale <code>/etc</code> , entonces <code>\$dir</code> se reemplaza por <code>/etc</code> antes de ejecutar la línea.
<code>"\$variable"</code>	Lo mismo, pero entre comillas — así, si el valor tiene espacios (ej. "Mis Documentos"), no se rompe en varias palabras.
<code>if / then / elif</code> <code>/ else / fi</code>	"Si pasa esto, hacé esto; si no, otra cosa". <code>fi</code> es "if" al revés — así bash sabe dónde termina el bloque.
<code>[condición]</code>	Es la forma de "preguntar algo" dentro de un <code>if</code> . Ojo: los espacios adentro de los corchetes son OBLIGATORIOS.
<code>-d "\$dir"</code>	Dentro de un <code>[]</code> : pregunta "¿existe ESTO y es un directorio?".
<code>while true; do</code> <code>... done</code>	Un bucle que se repite PARA SIEMPRE (porque la condición <code>true</code> nunca es falsa), hasta que algo adentro haga <code>exit</code> o <code>break</code> . Así se logra que el menú "no se cierre solo".
<code>case \$op in ...</code> <code>esac</code>	Como un menú de opciones: mira el valor de <code>\$op</code> y salta directo al bloque que coincide, en vez de encadenar muchos <code>if/elif</code> .
<code>1) comando ;;</code>	Dentro de un <code>case</code> : "si <code>op</code> es 1, ejecutá esto"; el <code>;;</code> marca el final de esa opción (como el <code>break</code> de otros lenguajes).
<code>*)</code>	Dentro de un <code>case</code> : la opción "cualquier otra cosa" (comodín), para cuando el usuario escribe algo inválido.
<code> </code> (pipe/tubería)	Toma la salida del comando de la izquierda y la usa como entrada del comando de la derecha. Permite encadenar varios comandos chiquitos

	para hacer algo más complejo.
<code>2>/dev/null</code>	"Los mensajes de error, tiralos a la basura" (<code>/dev/null</code> es un basurero que no guarda nada). Sirve para que errores de permisos no ensucien la pantalla.
<code>\$(comando)</code>	Ejecuta ese comando primero y reemplaza toda la expresión por lo que ese comando imprimió.

6.1 `mi_ls.sh` — listar con menú (ejercicio 1.1)

```
#!/bin/bash
clear
read -p "Directorio a analizar: " dir

if [ ! -d "$dir" ]; then
    echo "ERROR: '$dir' no es un directorio válido."
    exit 1
fi

while true; do
    clear
    echo "MENU - Listar archivos de: $dir"
    echo "1) Mas recientes (conteo por fecha)"
    echo "2) Mas antiguos (conteo por fecha)"
    echo "3) Mayor tamaño (conteo por tamaño)"
    echo "4) Menor tamaño (conteo por tamaño)"
    echo "5) Por tipo (archivo/directorio)"
    echo "6) Filtro: empieza con..."
    echo "7) Filtro: termina con..."
    echo "8) Filtro: contiene..."
    echo "9) Incluir subdirectorios (recursivo)"
    echo "0) Salir"
    read -p "Opcion: " op

    case $op in
        1) ls -lat "$dir" | awk '{print $6, $7}' | sort | uniq -c | less ;;
        2) ls -latr "$dir" | awk '{print $6, $7}' | sort | uniq -c | less ;;
        3) ls -laS "$dir" | awk '{print $5}' | sort -rn | uniq -c | less ;;
        4) ls -laSr "$dir" | awk '{print $5}' | sort -n | uniq -c | less ;;
        5) echo "Directorio: $dir"
            echo "--- Directorios (con ocultos):"; ls -la "$dir" | grep "^d" | wc -l
            echo "--- Archivos (con ocultos):"; ls -la "$dir" | grep -v "^d" | wc -l
            read -p "Enter para continuar..." ;;
        6) read -p "Cadena inicial: " patron
            find "$dir" -maxdepth 1 -name "$patron*" | less ;;
        7) read -p "Cadena final: " patron
            find "$dir" -maxdepth 1 -name "$patron*" | less ;;
        8) read -p "Cadena contenida: " patron
            find "$dir" -maxdepth 1 -name "$patron*" | less ;;
        9) read -p "Cadena a buscar (Enter = todos): " patron
            if [ -z "$patron" ]; then find "$dir" | less
            else find "$dir" -name "$patron*" | less; fi ;;
        0) echo "Chau!"; exit 0 ;;
        *) echo "Opcion invalida" ;;
    esac
done
```

Línea	Código	Explicación en criollo
1	#!/bin/bash	Avisa que este archivo es un script de bash.

2	<code>clear</code>	Limpia la pantalla antes de arrancar (pide el enunciado que se limpie antes de mostrar resultados).
3	<code>read -p "Directorio a analizar: " dir</code>	Pregunta qué directorio querés revisar y guarda la respuesta en la variable <code>dir</code> .
5-8	<code>if [! -d "\$dir"]; then ... exit 1; fi</code>	Chequeo de seguridad: si lo que escribiste NO es un directorio que existe, muestra un error y corta el script ahí mismo (no tiene sentido seguir).
10	<code>while true; do</code>	Arranca un bucle que se repite para siempre — así el menú vuelve a aparecer después de cada opción, hasta que el usuario elija salir.
11	<code>clear</code>	Limpia la pantalla en cada vuelta del menú, para que no se acumule texto viejo.
12-25	<code>echo "..."</code> (varias líneas)	Dibuja el menú en pantalla: título y las 10 opciones (1 a 9, más el 0 de salir).
26	<code>read -p "Opcion: " op</code>	Pregunta qué opción elegiste y la guarda en <code>op</code> .
28	<code>case \$op in</code>	Empieza a comparar el valor de <code>op</code> contra cada opción posible.
29	<code>1) ls -lat "\$dir" awk '{print \$6, \$7}' sort uniq -c less ;;</code>	Opción "más recientes": <code>ls -lat</code> lista con detalle (<code>-l</code>), incluyendo ocultos (<code>-a</code>), ordenado por fecha de modificación con lo más nuevo primero (<code>-t</code>). El resultado pasa por <code>awk</code> , que se queda solo con las columnas 6 y 7 de cada línea (que en <code>ls -l</code> son el mes y el día). Después <code>sort</code> pone juntas las fechas iguales, y <code>uniq -c</code> las junta en una sola línea contando cuántas veces se repite esa fecha — así se cumple lo de "indicar cuántos archivos comparten la misma fecha". <code>less</code> al final pagina el resultado si es muy largo.
30	<code>2) ls -latr "\$dir" awk ... sort uniq -c less ;;</code>	Opción "más antiguos": exactamente igual que la anterior, pero con <code>-r</code> agregado a <code>ls</code> , que invierte el orden (los más viejos quedan primero).
31	<code>3) ls -laS "\$dir" awk '{print \$5}' sort -rn uniq -c less ;;</code>	Opción "mayor tamaño": <code>-S</code> ordena por tamaño de archivo, de grande a chico. <code>awk '{print \$5}'</code> se queda con la columna 5 de <code>ls -l</code> (el tamaño en bytes). <code>sort -rn</code> ordena esos números de mayor a menor (<code>-n</code> =numérico, <code>-r</code> =al revés), y <code>uniq -c</code> cuenta cuántos archivos comparten exactamente el mismo tamaño.
32	<code>4) ls -laSr "\$dir" awk ... sort -n uniq -c less ;;</code>	Opción "menor tamaño": lo mismo pero invertido — del más chico al más grande.
33	<code>5) echo Directorio... ls -la grep "^d" wc -l</code>	Opción "por tipo": lista todo con <code>ls -la</code> . La primera columna de cada línea empieza con <code>d</code> si es un directorio, o con <code>-</code> si es un archivo normal. <code>grep "^d"</code> se queda solo con las líneas que EMPIEZAN (<code>^</code>) con <code>d</code> (los directorios) y <code>wc -l</code>

	<pre>... grep -v "^d" wc -l ...</pre>	las cuenta. <code>grep -v "^d"</code> hace lo opuesto (<code>-v</code> =invertir): se queda con todo lo que NO empieza con <code>d</code> , o sea los archivos, y también los cuenta.
34	<pre>6) read -p "Cadena inicial: " patron; find "\$dir" -maxdepth 1 -name "\$patron*" less ;;</pre>	Filtro "empieza con": pregunta el texto inicial. <code>find</code> busca dentro de <code>\$dir</code> (con <code>-maxdepth 1</code> = sin meterse en subcarpetas) archivos cuyo nombre empiece con ese texto — el <code>*</code> después del patrón significa "y cualquier cosa a continuación".
35	<pre>7) ... -name "*\$patron" less ;;</pre>	Filtro "termina con": el <code>*</code> ahora va ANTES del patrón, o sea "cualquier cosa, y después exactamente este texto al final".
36	<pre>8) ... -name "*\$patron*" less ;;</pre>	Filtro "contiene": el <code>*</code> a ambos lados — el texto puede estar en cualquier parte del nombre.
37	<pre>9) ... if [-z "\$patron"]; then find "\$dir" less; else find "\$dir" - name "\$patron*" less; fi ;;</pre>	Recursivo (con subcarpetas): a diferencia de las opciones 6-8, acá NO se pone <code>-maxdepth 1</code> , así que <code>find</code> se mete en todas las subcarpetas. <code>[-z "\$patron"]</code> pregunta "¿la respuesta quedó vacía?" (<code>-z</code> =zero length): si no escribiste nada, muestra todo; si escribiste algo, filtra por ese texto en cualquier parte del árbol de carpetas.
38	<pre>0) echo "Chau!"; exit 0 ;;</pre>	Sale del script (y por lo tanto del bucle <code>while true</code>).
39	<pre>*) echo "Opcion invalida" ;;</pre>	Si escribiste algo que no es ninguna opción del menú, avisa y vuelve a mostrar el menú (porque el <code>while true</code> sigue).
40-41	<pre>esac / done</pre>	Cierran el <code>case</code> y el <code>while</code> , respectivamente.

Nota sobre `less`: es un "paginador" — si el resultado es muy largo para la pantalla, permite moverse con las flechas o la barra espaciadora en vez de que todo se desborde de una. Cumple el requisito del enunciado de paginar salidas extensas.

Resultado real de ejecución (opción 5, sobre `/etc`): 112 directorios y 172 archivos, incluyendo ocultos.

6.2 `buscar.sh` — búsqueda y visualización (ejercicio 1.2)

```

#!/bin/bash
while true; do
    clear
    echo "MENU - Búsqueda y visualización"
    echo "1) Buscar archivo (o parte del nombre) en un directorio"
    echo "2) Buscar palabra (o parte) dentro de un archivo"
    echo "3) Buscar archivos y buscar una palabra dentro de ellos"
    echo "4) Contar líneas de un archivo"
    echo "5) Mostrar las primeras n líneas"
    echo "6) Mostrar las últimas n líneas"
    echo "0) Salir"
    read -p "Opción: " op

    case $op in
        1) read -p "Directorio: " dir
            read -p "Nombre (o parte): " patron
            echo "--- Resultados:"; find "$dir" -name "$patron" 2>/dev/null
            echo "--- Total de ocurrencias:"; find "$dir" -name "$patron"
2>/dev/null | wc -l
            read -p "Enter para continuar..." ;;
        2) read -p "Archivo: " archivo
            read -p "Palabra (o parte): " palabra
            echo "--- Coincidencias (línea: contenido):"; grep -n "$palabra"
"$archivo" 2>/dev/null
            echo "--- Total de ocurrencias:"; grep -c "$palabra" "$archivo"
2>/dev/null
            read -p "Enter para continuar..." ;;
        3) read -p "Directorio: " dir
            read -p "Nombre de archivo (o parte): " patron_archivo
            read -p "Palabra a buscar dentro: " palabra
            for f in $(find "$dir" -name "$patron_archivo*" 2>/dev/null); do
                echo "== $f =="; grep -n "$palabra" "$f" 2>/dev/null
                echo "    Total: $(grep -c "$palabra" "$f" 2>/dev/null)"
            done
            read -p "Enter para continuar..." ;;
        4) read -p "Archivo: " archivo; wc -l "$archivo"
            read -p "Enter para continuar..." ;;
        5) read -p "Archivo: " archivo; read -p "Número de líneas (n): " n
            head -n "$n" "$archivo"; read -p "Enter para continuar..." ;;
        6) read -p "Archivo: " archivo; read -p "Número de líneas (n): " n
            tail -n "$n" "$archivo"; read -p "Enter para continuar..." ;;
        0) echo "Chau!"; exit 0 ;;
        *) echo "Opción inválida" ;;
    esac
done

```

Línea	Código	Explicación en criollo
1	#!/bin/bash	Es un script de bash.

2	<code>while true; do</code>	El menú se repite para siempre hasta que se elija salir.
3	<code>clear</code>	Limpia pantalla en cada vuelta.
4-11	<code>echo "... " ×7</code>	Dibuja las 7 líneas del menú (título + 6 opciones + salir).
12	<code>read -p "Opcion: " op</code>	Guarda la opción elegida en <code>op</code> .
14	<code>case \$op in</code>	Empieza a mirar qué opción es.
15-19	<code>1) read -p "Directorio: " dir; read -p "Nombre (o parte): " patron; echo "--- Resultados:"; find "\$dir" -name "\$patron" 2>/dev/null; echo "--- Total..."; find ... wc -l</code>	Buscar archivo por nombre: pide directorio y el texto a buscar. <code>find</code> busca archivos cuyo nombre CONTenga ese texto (los <code>*</code> a los dos lados). <code>2>/dev/null</code> esconde errores tipo "permiso denegado" en carpetas que no podés leer. Se ejecuta el mismo <code>find</code> dos veces: la primera para MOSTRAR los archivos encontrados, la segunda con <code> wc -l</code> al final para CONTAR cuántos fueron — así se cumple "mostrar dónde están y el total de ocurrencias" que pide el enunciado.
20-24	<code>2) read -p "Archivo: " archivo; read -p "Palabra..." palabra; grep -n "\$palabra" "\$archivo"; grep -c "\$palabra" "\$archivo"</code>	Buscar palabra dentro de un archivo: pide el archivo y la palabra. <code>grep -n</code> muestra CADA línea donde aparece la palabra, con su número de línea antepuesto (<code>-n</code>). <code>grep -c</code> en vez de mostrar las líneas, las CUENTA — da el total de ocurrencias.
26-32	<code>3) read dir; read patron_archivo; read palabra; for f in \$(find "\$dir" -name "\$patron_archivo*"); do echo "== \$f =="; grep -n "\$palabra" "\$f"; echo "Total: \$(grep -c "\$palabra" "\$f")"; done</code>	Combinado (buscar archivo Y palabra adentro): primero <code>find</code> encuentra los archivos que coinciden con el nombre, y el <code>for f in \$(...)</code> los recorre uno por uno (la variable <code>f</code> toma, en cada vuelta, la ruta de un archivo distinto). Por cada archivo encontrado: imprime su nombre como encabezado, busca la palabra adentro con <code>grep -n</code> (mostrando línea y número), y al final imprime cuántas veces apareció con <code>grep -c</code> metido dentro de <code>\$(...)</code> (para insertar ese número directo en el <code>echo</code>).
34-35	<code>4) read -p "Archivo: " archivo; wc -l "\$archivo"</code>	Contar líneas: <code>wc -l</code> ("word count, líneas") cuenta cuántas líneas tiene el archivo.
37-38	<code>5) read archivo; read n; head -n "\$n" "\$archivo"</code>	Primeras n líneas: <code>head -n N</code> muestra solo las primeras N líneas del archivo (el principio).
40-41	<code>6) read archivo; read n; tail -n "\$n" "\$archivo"</code>	Últimas n líneas: <code>tail -n N</code> muestra solo las últimas N líneas (el final).
43	<code>0) echo "Chau!"; exit 0 ;;</code>	Sale del script.

44	<code>*) echo "Opcion invalida" ;;</code>	Cualquier otra tecla: avisa y vuelve a mostrar el menú.
----	---	---

Resultado real: buscando `dev` en `/etc/fstab` aparecieron 3 líneas (`/dev/sda1`, `/dev/sda2`, `/dev/sda3`) con su número de línea.

6.3 `revisar_logs.sh` — revisión de logs (ejercicio 1.3)

```
#!/bin/bash
LOG1="/var/log/syslog"
LOG2="/var/log/messages"
LOG3="/var/log/secure"

while true; do
    clear
    echo "MENU - Revision de logs"
    echo "1) Mostrar las ultimas 15 lineas de los 3 logs"
    echo "2) Filtrar esas lineas por una palabra"
    echo "0) Salir"
    read -p "Opcion: " op

    case $op in
        1) for log in "$LOG1" "$LOG2" "$LOG3"; do
            echo "==== $log ====="
            tail -n 15 "$log" 2>/dev/null || echo "(no existe o sin permisos)"
        done
        read -p "Enter para continuar..." ;;
        2) read -p "Palabra a filtrar: " palabra
            for log in "$LOG1" "$LOG2" "$LOG3"; do
                echo "==== $log (filtro: $palabra) ====="
                tail -n 15 "$log" 2>/dev/null | grep "$palabra" || echo "(sin coincidencias)"
            done
            read -p "Enter para continuar..." ;;
        0) echo "Chau!"; exit 0 ;;
        *) echo "Opcion invalida" ;;
    esac
done
```

Línea	Código	Explicación en criollo
2-4	<code>LOG1="/var/log/syslog"</code> (×3)	Guarda en 3 variables las rutas de los 3 archivos de log que vamos a revisar. Así, si hay que cambiar una ruta, se cambia en un solo lugar.
6	<code>while true; do</code>	Menú que se repite hasta salir.
7	<code>clear</code>	Limpia pantalla (lo pide el enunciado: "clear the screen before displaying results").

8-12	<code>echo "... " ×5</code>	Dibuja el menú: título + 2 opciones + salir.
13	<code>read -p "Opcion: " op</code>	Guarda la elección.
15-19	<code>1) for log in "\$LOG1" "\$LOG2" "\$LOG3"; do echo "==== \$log" "===="; tail -n 15 "\$log" 2>/dev/null echo "(no existe...)"; done</code>	Mostrar últimas 15 líneas de los 3 logs: el <code>for log in ...</code> recorre las 3 rutas una por una (en cada vuelta, <code>\$log</code> es una ruta distinta). Por cada una: imprime un encabezado, y <code>tail -n 15</code> muestra las últimas 15 líneas de ESE archivo (lo más reciente, porque los logs se escriben agregando al final). El <code> echo "... "</code> es un "por si falla": si el archivo no existe o no hay permiso, en vez de romperse, muestra un aviso.
21-27	<code>2) read -p "Palabra a filtrar: " palabra; for log in ...; do echo "..."; tail -n 15 "\$log" 2>/dev/null grep "\$palabra" echo "(sin coincidencias)"; done</code>	Filtrar esas líneas por una palabra: pide la palabra a buscar. Por cada log, toma sus últimas 15 líneas (<code>tail -n 15</code>) y de esas, con <code> </code> , se queda solo con las que contienen la palabra (<code>grep</code>). Si ninguna línea de esas 15 tiene la palabra, avisa "sin coincidencias" en vez de mostrar nada vacío.
29-30	<code>0) echo "Chau!"; exit 0 ;; / *) echo "Opcion invalida" ;;</code>	Salir / opción inválida, igual que en los scripts anteriores.

Resultado real (filtro `sshd` sobre `/var/log/messages`): conexiones/desconexiones reales del usuario `vagrant` vía SSH con autenticación de clave pública.

6.4 `newgroup.sh` y `newuser.sh` (ejercicio 1.4)

```
#!/bin/bash
# newgroup.sh
if [ $# -ne 2 ]; then
    echo "Uso: $0 "
    exit 1
fi
GRUPO="$1"; GID="$2"

if getent group "$GID" > /dev/null 2>&1; then
    echo "ERROR: el GID $GID ya existe en el sistema."
    exit 1
fi
groupadd -g "$GID" "$GRUPO" && echo "Grupo $GRUPO creado (GID $GID)"
```

```
#!/bin/bash
# newuser.sh
if [ $# -ne 8 ]; then
    echo "Uso: $0 usuario grupo \"nombre\" home shell perm_home perm_dir1 perm_dir2"
    exit 1
fi
USER="$1"; GRUPO="$2"; NOMBRE="$3"; HOME_DIR="$4"; SHELL="$5"; P1="$6"; P2="$7";
P3="$8"

if ! getent group "$GRUPO" > /dev/null 2>&1; then
    groupadd "$GRUPO"
fi
if id "$USER" > /dev/null 2>&1; then
    echo "ERROR: el usuario $USER ya existe."; exit 1
fi
useradd -d "$HOME_DIR" -m -c "$NOMBRE" -g "$GRUPO" -s "$SHELL" "$USER"
mkdir -p "$HOME_DIR/documentos" "$HOME_DIR/scripts"
chmod "$P1" "$HOME_DIR"
chmod "$P2" "$HOME_DIR/documentos"
chmod "$P3" "$HOME_DIR/scripts"
```

newgroup.sh línea por línea

Línea	Código	Explicación en criollo
3-6	<pre>if [\$# -ne 2]; then echo "Uso: ..."; exit 1; fi</pre>	<code>\$#</code> es "cuántos argumentos me pasaron al llamar el script" (ej. <code>./newgroup.sh developers 1001</code> son 2). <code>-ne 2</code> = "distinto de 2". Si no le diste exactamente 2 datos, muestra cómo se usa y corta.
7	<pre>GRUPO="\$1"; GID="\$2"</pre>	<code>\$1</code> es el primer argumento (nombre del grupo), <code>\$2</code> el segundo (el GID/número). Los guarda en variables con nombre más claro.
9-12	<pre>if getent group "\$GID" > /dev/null 2>&1; then echo "ERROR..."; exit 1; fi</pre>	<code>getent group</code> le pregunta al sistema si YA existe un grupo con ese GID. <code>> /dev/null 2>&1</code> esconde toda su salida (solo nos importa si "encontró algo" o no, no lo que imprime). Si ya existe, avisa error y corta — así se evita crear un GID duplicado.
13-14	<pre>groupadd -g "\$GID" "\$GRUPO" && echo "Grupo creado" echo "ERROR..."</pre>	<code>groupadd -g</code> crea el grupo con ESE número de GID exacto (si no se pone <code>-g</code> , el sistema elegiría el siguiente disponible solo). El <code>&&</code> significa "si lo anterior salió bien, hacé esto"; el <code> </code> es "si salió mal, hacé esto otro" — es un if/else compacto en una sola línea.

newuser.sh línea por línea

Línea	Código	Explicación en criollo
-------	--------	------------------------

3-6	<pre>if [\$# -ne 8]; then echo "Uso: ..."; exit 1; fi</pre>	Este script necesita 8 datos exactos (usuario, grupo, nombre, home, shell, y 3 permisos). Si no vinieron los 8, muestra el formato correcto y corta.
10	<pre>USER="\$1"; GRUPO="\$2"; NOMBRE="\$3"; HOME_DIR="\$4"; SHELL="\$5"; P1="\$6"; P2="\$7"; P3="\$8"</pre>	Toma cada uno de los 8 argumentos posicionales y les pone un nombre entendible (en vez de acordarse "el \$4 es el home", ahora es \$HOME_DIR).
13-15	<pre>if ! getent group "\$GRUPO" > /dev/null 2>&1; then groupadd "\$GRUPO"; fi</pre>	El ! antes de getent lo niega: "si el grupo NO existe todavía, entonces créalo". Así, si el grupo ya existía de antes, no intenta crearlo de nuevo (lo cual daría error).
17-20	<pre>if id "\$USER" > /dev/null 2>&1; then echo "ERROR: ya existe"; exit 1; fi</pre>	id usuario consulta si ese usuario ya existe en el sistema. Si existe, avisa y corta — evita crear un usuario duplicado.
21	<pre>useradd -d "\$HOME_DIR" -m -c "\$NOMBRE" -g "\$GRUPO" -s "\$SHELL" "\$USER"</pre>	Crea el usuario de verdad: -d fija dónde va su carpeta personal, -m hace que esa carpeta se cree físicamente (si no, quedaría solo "anotado" pero sin carpeta), -c guarda el nombre completo (el campo "comentario"/GECOS), -g lo mete en ese grupo como grupo principal, -s define qué shell usa al iniciar sesión (ej. /bin/bash).
24	<pre>mkdir -p "\$HOME_DIR/documentos" "\$HOME_DIR/scripts"</pre>	Crea dos subcarpetas dentro del home del usuario. -p hace que no falle si alguna carpeta intermedia no existía todavía.
25-27	<pre>chmod "\$P1" "\$HOME_DIR"; chmod "\$P2" ../documentos; chmod "\$P3" ../scripts</pre>	Aplica los 3 permisos que pediste (en formato octal, ej. 700, 770, 755) a la carpeta home y a cada subcarpeta. Cada dígito octal = dueño/grupo/otros, sumando r=4, w=2, x=1.

Ejemplo de uso y resultado real:

```
$ ./newuser.sh alice developers "Alice Developer" /home/alice /bin/bash 700 770 755
Usuario alice creado (home: /home/alice, grupo: developers, shell: /bin/bash)
Permisos aplicados: /home/alice=700, documentos=770, scripts=755

$ ls -la /home/alice/
drwx----- 4 alice developers ...          <- 700: solo alice entra
drwxrwx--- 2 root  root      ... documentos <- 770: dueño+grupo, nada para otros
drwxr-xr-x 2 root  root      ... scripts    <- 755: dueño todo, resto solo
lectura/ejecucion

$ ./newuser.sh alice developers "Alice" /home/alice /bin/bash 700 770 755
ERROR: el usuario alice ya existe. <- evita duplicados
```

7. Editor VI — cómo usarlo paso a paso

La idea que hay que tener clarísima antes de tocar VI

VI tiene **3 modos**, y todo lo raro que le pasa a un principiante (que no puede escribir, que borra cosas solo, que no puede salir) es porque no sabe en qué modo está:

Modo	¿Para qué sirve?	¿Cómo entro?	¿Cómo salgo?
Normal	Moverse, borrar, copiar, pegar. Es el modo en el que VI arranca SIEMPRE.	Es el inicial (o presionando <code>Esc</code> desde cualquier otro modo)	—
Inserción	Escribir texto de verdad, como en un Bloc de notas.	<code>i</code> (insertar antes del cursor) o <code>a</code> (después)	<code>Esc</code>
Comando	Guardar, salir, buscar, reemplazar texto.	<code>:</code> (dos puntos)	<code>Enter</code> (ejecuta) o <code>Esc</code> (cancela)

Regla de oro: las teclas en modo Normal son *verbos* (borrar, copiar, mover), no letras que se escriben en el documento.

7.1 Tutorial paso a paso: crear y editar `himno.txt`

1. **Abrir/crear el archivo:** en la terminal, escribís `vi himno.txt` y `Enter`. Si el archivo no existe, VI abre una pantalla vacía lista para crearlo (recién se crea en el disco cuando lo guardes).
2. **Entrar a modo Inserción:** presionás `i`. Vas a ver algo como `-- INSERT --` abajo a la izquierda — esa es la señal de que ya podés escribir normal.
3. **Escribir el texto:** tipeás cada línea del himno, presionando **Enter** al final de cada una para pasar a la siguiente línea (el enunciado pide que cada línea del texto quede en una línea separada del editor).
4. **Volver a modo Normal:** cuando termines de escribir, presionás `Esc`. El `-- INSERT --` de abajo desaparece — ya estás en modo Normal de nuevo.
5. **Guardar sin salir:** escribís `:w` y `Enter`. El archivo queda guardado en el disco, pero seguís adentro de VI editando.
6. **Hacer un reemplazo en un rango de líneas:** `:1,4s/a/-/g` + `Enter` — cambia todas las "a" de las líneas 1 a 4 por guiones.
7. **Hacer un reemplazo en TODO el archivo:** `:%s/a/##/g` + `Enter`.
8. **Borrar una palabra:** ponés el cursor arriba de la palabra (con las flechas, en modo Normal) y presionás `dw`.
9. **Borrar varias líneas de una:** `:13,16d` + `Enter` borra las líneas 13 a 16 completas.

10. **Deshacer:** si te arrepentís de lo último que hiciste, en modo Normal presionás `u` (una vez por cada paso que quieras deshacer).
11. **Ir al final del archivo y poner esa línea en mayúsculas:** `G` (te lleva a la última línea) y después `gUU` (convierte toda esa línea a mayúsculas).
12. **Copiar y pegar líneas:** parado en una línea, `yy` la copia; movés el cursor a donde querés pegarla y presionás `p` (pega DESPUÉS de la línea actual).
13. **Buscar una palabra:** `/Escuela` + Enter te lleva a la primera aparición; presionando `n` saltás a la siguiente.
14. **Ir directo a una línea:** `5G` te para justo en la línea 5.
15. **Guardar y salir en un solo paso:** `:wq` + Enter.
16. **Reabrir el archivo y borrar las primeras 5 líneas:** `vi himno.txt` de nuevo, y ya adentro `:1,5d` + Enter.
17. **Salir SIN guardar los cambios:** `:q!` + Enter — el `!` fuerza la salida descartando lo que hayas tocado desde el último `:w`.

7.2 Tabla de referencia rápida (todos los comandos usados)

Comando	Desglose	Qué hace
<code>vi</code> <code>himno.txt</code>	abre el editor con ese archivo (lo crea si no existe)	Iniciar edición
<code>i</code> / <code>a</code>	insert / append	Insertar antes / después del cursor
<code>Esc</code>	—	Volver a modo Normal
<code>:w</code>	write	Guardar sin salir
<code>:1,4s/a/-/g</code>	<code>1,4</code> =rango · <code>s</code> =substitute · <code>/a/</code> =patrón · <code>-</code> =reemplazo · <code>g</code> =global (todas por línea)	Reemplazar "a" por "-" en las líneas 1 a 4
<code>:%s/al/##/g</code>	<code>%</code> = todo el archivo (equivale a <code>1,\$</code>)	Reemplazar "al" por "##" en todo el documento
<code>dw</code>	delete + word (verbo+objeto)	Borrar la palabra bajo el cursor
<code>:13,16d</code>	rango + delete	Borrar las líneas 13 a 16
<code>u</code>	undo	Deshacer el último cambio
<code>G</code>	go to end	Ir a la última línea
<code>gUU</code>	gU (uppercase) repetido = línea completa	Convertir la línea actual a mayúsculas
<code>7G</code> / <code>5G</code>	número + go to	Ir a la línea 7 / línea 5
<code>yy</code> / <code>p</code>	yank / paste	Copiar línea / pegar después
<code>/Escuela</code>	búsqueda hacia adelante	Buscar "Escuela" (<code>n</code> = siguiente coincidencia)

<code>:wq</code>	write + quit	Guardar y salir
<code>:q!</code>	quit + forzar	Salir sin guardar
<code>:1,5d</code>	rango + delete	Borrar las 5 primeras líneas al reabrir
<code>:set number</code>	—	Mostrar números de línea a la izquierda

Guía comando por comando aún más detallada (con "cómo explicarlo" y preguntas típicas) en [maquinas/GUIA-SUSTENTACION-LAB02.pdf](#), Parte 1.

8. SAMBA / SMB — configuración y comandos

Servidor SAMBA clásico en Solaris 11.4 (paquete `service/network/samba` 4.7.6), compartiendo el recurso `compartido` en el puerto **445/TCP**.

```
[global]
workgroup = WORKGROUP
security = user
passdb backend = tdbsam
map to guest = Bad User
server min protocol = SMB2

[compartido]
path = /export/compartido
writable = yes
valid users = claudia admin
create mask = 0660
directory mask = 0770
```

Comando	Explicación
<code>smbpasswd -a claudia</code>	-a =add: crea la cuenta SMB de un usuario Unix ya existente, con contraseña independiente de la del sistema.
<code>pdbedit -L</code>	Lista los usuarios de la base SMB.
<code>testparm</code>	Valida la sintaxis de <code>smb.conf</code> antes de reiniciar el servicio.
<code>netstat -an grep .445</code>	Confirma que el servicio esté escuchando en el puerto SMB.
<code>smbclient -L //IP -U usuario</code>	Lista los shares disponibles en el servidor (cliente Unix).
<code>smbclient //IP/compartido -U usuario -c 'put x; get y'</code>	Sube (<code>put</code>) y baja (<code>get</code>) archivos sin entrar a la sesión interactiva.
<code>net use \\IP\compartido /user:usuario clave</code>	Mapea la unidad de red desde Windows (ruta UNC).
<code>smbstatus</code>	Muestra quién está conectado en ese momento al servidor.

Desglose comando por comando de TODOS los parámetros de SAMBA en `maquinas/GUIA-SUSTENTACION-LAB02.pdf` , Parte 2.

9. Despliegue de VMs y redes

Se clonaron las 3 VMs base (Slackware 15.0, Solaris 11.4, Windows Server GUI). Cada clon tiene un script de auto-detección de red al arranque:

```
# Lógica de auto-deteccion (simplificada)
if ping -c1 10.2.65.1 &> /dev/null; then
    # Estamos en la universidad: IP estatica
    ip addr add 10.2.78.7X/24 dev eth0
else
    # Estamos en casa: DHCP
    dhclient eth0
fi
```

Matriz de pings verificada (casa, 0% de pérdida en todas las pruebas):

Origen → Destino	Comando	Resultado
Slackware → Solaris	ping -c 2 192.168.1.11	2/2, rtt ~1.6 ms
Solaris → Windows	ping 192.168.1.14 2	alive
Windows → Slackware	ping -n 2 192.168.1.13	2/2, 0% loss
Cualquiera → Internet	ping 8.8.8.8	2/2, 0% loss

10. Conceptos raros / preguntas trampa

OUI (Organizationally Unique Identifier). Los primeros 3 bytes de una MAC identifican al fabricante. `08:00:27` es el OUI de VirtualBox — por eso, aunque una VM reporte "1000 Mb/s full-duplex", su MAC delata que la NIC es virtual, no física.

ARP (Address Resolution Protocol). Traduce IP (capa 3) a MAC (capa 2) dentro de la misma red local. Sin ARP, un dispositivo sabría la IP destino pero no podría armar la trama Ethernet (que necesita la MAC).

TTL (Time To Live). Contador que decrementa 1 en cada router que atraviesa un paquete IP; si llega a 0 se descarta. Evita loops infinitos de enrutamiento. Un TTL de 128 típico de Windows, 64 típico de Linux/Unix — a veces delata el sistema operativo de origen.

Syslog (RFC 5424). Estándar de logging: define *facilidad* (qué subsistema genera el mensaje: kernel, mail, auth...) + *severidad* (emergencia, error, info...), transporte por UDP 514, y un formato de mensaje con timestamp + host + proceso[pid]. Windows usa su propio Event Log (no es syslog nativo).

DCE/DTE en un cable serial. En un enlace WAN punto a punto, un extremo debe "marcar el ritmo" de la comunicación (clock rate) — ese es el DCE (Data Communications Equipment); el otro extremo (DTE, Data Terminal Equipment) sincroniza con ese reloj. En Packet Tracer, el cable rojo serial asigna automáticamente cuál extremo es cuál.

Máscaras octales de permisos (chmod). Cada dígito representa dueño/grupo/otros, y cada uno se arma sumando $r=4$, $w=2$, $x=1$. $7=rwx$ ($4+2+1$), $6=rw-$ ($4+2$), $5=r-x$ ($4+1$), 0 =sin permisos. Por eso `0770` = dueño y grupo con todo, "otros" sin nada.

TLS / QUIC / HTTP en la misma captura. HTTP (puerto 80) viaja en texto plano — se lee el contenido. HTTPS/TLS (puerto 443, TCP) cifra el contenido; solo se ve el handshake. QUIC es HTTP/3: corre sobre UDP en vez de TCP (más rápido para establecer conexión, usado por Chrome con Google/YouTube).

Retransmisión TCP vs ACK duplicado. Si un segmento se pierde, el emisor lo reenvía tras no recibir ACK a tiempo (retransmisión). Si el receptor nota un hueco en la secuencia, reenvía el mismo ACK varias veces para "avisar" que le falta algo (ACK

duplicado) — ambos son mecanismos normales de TCP para garantizar entrega confiable, no son errores graves si son pocos.

¿Por qué Wireshark y Packet Tracer muestran la MISMA encapsulación?

Porque ambos respetan el modelo real de redes (OSI/TCP-IP): Packet Tracer la SIMULA con dispositivos virtuales, Wireshark la CAPTURA de tráfico real — pero la estructura de capas (Ethernet→IP→TCP/ICMP→datos) es idéntica en los dos, porque es el mismo estándar el que la define, no el programa.